3. Java: simple explanations and professional answers

1. Object-oriented design, interfaces and composition [A]

Plain English: An object keeps related data and behavior together. Encapsulation protects its rules. An interface describes what a component can do. Polymorphism lets different implementations satisfy that interface. Composition means using another component; inheritance means being a specialized version of it.

Say it professionally: "I use interfaces to define contracts and keep implementations replaceable. I prefer composition when I need flexible behavior, and inheritance when the subtype truly satisfies the parent contract. I keep business rules inside the component responsible for them."

Example and follow-up: A PaymentService depends on a PaymentGateway interface. Tests inject a fake gateway. Explain how Strategy selects a payment method and why a class with many unrelated responsibilities is hard to test. Know overloading versus overriding, interface versus abstract class, and checked versus unchecked exceptions.

2. SOLID and practical design patterns [A]

Plain English: SOLID encourages small responsibilities, replaceable implementations, and dependencies on useful contracts. It is a guide to controlling change, not a requirement to create many classes.

Say it professionally: "I apply design principles where they reduce coupling and make change safer. For example, a strategy separates payment-specific rules from orchestration, while an adapter hides the details of an external provider. I avoid adding abstractions without a clear need."

Know: Single responsibility; open for extension without repeated edits; substitutable subtypes; small client-focused interfaces; dependency inversion. Factory creates implementations; Builder constructs complex values; Decorator adds behavior. Dependency injection is how dependencies are supplied, not a substitute for good boundaries.

SOLID principle	Plain English	In the gateway example below
S - Single Responsibility	Give a class one main reason to change.	`PaymentService` handles the business step; each gateway translates for one provider.
O - Open/Closed	Add behavior without repeatedly changing stable processing logic.	Add `BankBGateway`; keep `PaymentService.process` unchanged.
L - Liskov Substitution	A replacement must honor the parent type's contract.	Both gateways must return a receipt on success and report failure according to the same agreed rules. This applies to interface implementations as well as class inheritance.
I - Interface Segregation	Give callers only the operations they need.	The service needs a small `charge` contract, not every operation in a bank SDK.
D - Dependency Inversion	Business code depends on a contract it controls.	`PaymentService` depends on `PaymentGateway`; provider-specific code implements it.

Composition versus inheritance: Composition means a class *has* another object. `PaymentService` has a `PaymentGateway`, which can be replaced. Inheritance means a class *is a specialized kind of* another class. A `WirePayment` might extend an abstract `Payment` only if it can fulfill every rule promised by `Payment`; if it must disable a parent operation, reconsider the hierarchy. Open/Closed is a design goal that either technique can support. Liskov applies to every subtype, including a class that implements an interface.

Complete example: adding Bank B without changing PaymentService

Save the following as `GatewayDemo.java`. It is one runnable Java 8 class. Both bank clients simulate external APIs; there are no real charges or network calls. The different request and response shapes are intentional: each gateway hides its bank's details from the service.

```
import java.math.BigDecimal;
import java.math.RoundingMode;

public class GatewayDemo {
    static class Payment {
        final BigDecimal amount;
        Payment(String amount) {
            this.amount = new BigDecimal(amount)
                    .setScale(2, RoundingMode.UNNECESSARY);
            if (this.amount.signum() <= 0)
                throw new IllegalArgumentException("Amount must be positive");
        }
    }

    static class Receipt {
        final String reference;
        Receipt(String reference) { this.reference = reference; }
    }

    interface PaymentGateway {
        Receipt charge(Payment payment);
    }

    static class BankAResponse {
        final String transactionId;
        BankAResponse(String id) { this.transactionId = id; }
    }

    static class BankAClient {
        BankAResponse submitPayment(BigDecimal amount) {
            System.out.println("Bank A received $" + amount);
            return new BankAResponse("A-1001");
        }
    }

    static class BankBClient {
        String createCharge(long amountInCents) {
            System.out.println("Bank B received " + amountInCents + " cents");
            return "B-2001";
        }
    }

    static class BankAGateway implements PaymentGateway {
        private final BankAClient client;
        BankAGateway(BankAClient client) { this.client = client; }
        public Receipt charge(Payment payment) {
            BankAResponse response = client.submitPayment(payment.amount);
            return new Receipt(response.transactionId);
```

```
        }
    }

    // New provider adapter: Bank B's API uses cents and returns a String.
    static class BankBGateway implements PaymentGateway {
        private final BankBClient client;
        BankBGateway(BankBClient client) { this.client = client; }
        public Receipt charge(Payment payment) {
            long cents = payment.amount.movePointRight(2).longValueExact();
            String reference = client.createCharge(cents);
            return new Receipt(reference);
        }
    }

    static class PaymentService {
        private final PaymentGateway gateway;
        PaymentService(PaymentGateway gateway) { this.gateway = gateway; }
        Receipt process(Payment payment) {
            return gateway.charge(payment);
        }
    }

    public static void main(String[] args) {
        PaymentService bankAService = new PaymentService(
                new BankAGateway(new BankAClient()));
        Receipt first = bankAService.process(new Payment("10.50"));
        System.out.println("Receipt: " + first.reference);

        PaymentService bankBService = new PaymentService(
                new BankBGateway(new BankBClient()));
        Receipt second = bankBService.process(new Payment("7.25"));
        System.out.println("Receipt: " + second.reference);
    }
}
```

Run `javac GatewayDemo.java` and then `java GatewayDemo`. You should see Bank A receive $10.50 and return `A-1001`, then Bank B receive 725 cents and return `B-2001`. These are two separate simulated payments. The clients (`BankAClient` and `BankBClient`) represent provider-specific SDK or HTTP code; the gateway adapters translate their results into our `Receipt`.

What changed to add Bank B: Add `BankBClient` and `BankBGateway`, then choose the new gateway when constructing the service. `PaymentService` still calls `gateway.charge(payment)`. In a Spring application, configuration or dependency injection would do the construction. The provider's contract must still define failure and retry behavior; this short example demonstrates structure, not production payment safety.

Hexagonal architecture connection: The application owns the `PaymentGateway` port. A bank gateway is an external adapter that implements it. The source-code dependency points toward the application contract, even though a payment call travels from the service outward to the bank. This is dependency inversion across architectural layers, not simply another name for composition.

Professional answer to practice: "I use composition to give the payment service a gateway through a small interface. Adding a bank means implementing another adapter and selecting it at application setup. The service stays focused on the payment flow. Each adapter must honor the gateway contract so it can safely replace another implementation."

Strategy pattern: a runnable payment example

Plain English: The overall payment process is the same, but wire and check payments have different limits and fees. A *strategy* owns those changing rules. `PaymentService` selects a strategy and handles steps that are common to every payment. In this example it checks for a positive amount and calculates the total; saving and submitting a real payment are outside this small demonstration.

```
import java.math.BigDecimal;
import java.util.EnumMap;
import java.util.Map;

public class StrategyExample {
    enum PaymentType { WIRE, CHECK }

    interface PaymentStrategy {
        void validate(BigDecimal amount);
        BigDecimal calculateFee(BigDecimal amount);
    }

    static class WireStrategy implements PaymentStrategy {
        public void validate(BigDecimal amount) {
            if (amount.compareTo(new BigDecimal("10000")) > 0) {
                throw new IllegalArgumentException("Wire exceeds the limit");
            }
        }

        public BigDecimal calculateFee(BigDecimal amount) {
            return new BigDecimal("15.00");
        }
    }

    static class CheckStrategy implements PaymentStrategy {
        public void validate(BigDecimal amount) {
            if (amount.compareTo(new BigDecimal("5000")) > 0) {
                throw new IllegalArgumentException("Check exceeds the limit");
            }
        }

        public BigDecimal calculateFee(BigDecimal amount) {
            return new BigDecimal("2.00");
        }
    }

    static class PaymentService {
        private final Map<PaymentType, PaymentStrategy> strategies =
                new EnumMap<>(PaymentType.class);

        PaymentService() {
            strategies.put(PaymentType.WIRE, new WireStrategy());
            strategies.put(PaymentType.CHECK, new CheckStrategy());
        }

        BigDecimal process(PaymentType type, BigDecimal amount) {
            if (type == null || amount == null ||
                    amount.compareTo(BigDecimal.ZERO) <= 0) {
```

```
                throw new IllegalArgumentException("Invalid payment");
            }

            PaymentStrategy strategy = strategies.get(type);
            strategy.validate(amount);
            BigDecimal total = amount.add(strategy.calculateFee(amount));

            // Common steps in a real service: save, submit, record the outcome.
            return total;
        }
    }

    public static void main(String[] args) {
        PaymentService service = new PaymentService();
        System.out.println(service.process(
                PaymentType.WIRE, new BigDecimal("100.00"))); // 115.00
        System.out.println(service.process(
                PaymentType.CHECK, new BigDecimal("100.00"))); // 102.00
    }
}
```

How to explain it: `PaymentStrategy` defines the operations that vary. `WireStrategy` and `CheckStrategy` implement their own limits and fees. `PaymentService` chooses the right implementation and keeps the shared flow in one place. To add a new payment type, add its strategy and register it. An *adapter* would be a separate component that translates our request into an external provider's format.

Professional answer to practice: "I use the Strategy pattern when several payment types share an overall process but have different business rules. The service selects a strategy, and that strategy handles type-specific validation and fee calculation. This keeps the common payment flow easier to read and test."

Other design principles: KISS, YAGNI, DRY and more [A]

These are decision guides, not absolute laws. In an interview, give the meaning, a concrete example, and one tradeoff. You probably mean **YAGNI**, pronounced "yag-nee": **You Aren't Gonna Need It**.

Principle	Plain English	Professional interview line
KISS - Keep It Simple	Choose the simplest design that meets the real requirements. Simple does not mean skipping correctness, security, or tests.	"I start with a clear implementation that meets today's requirements, then add complexity only when it solves a demonstrated problem."
YAGNI - You Aren't Gonna Need It	Do not build features or extension points for a future need that nobody has confirmed.	"I avoid speculative abstractions. I keep the code easy to change, and introduce a new layer when there is a concrete second use case or requirement."
DRY - Don't Repeat Yourself	Keep each business rule in one authoritative place so changes do not require inconsistent edits. Similar-looking code is not always the same rule.	"I remove duplicated knowledge, but I avoid forcing unrelated workflows into one abstraction just because they share a few lines."
Separation of concerns	Give different responsibilities clear homes: HTTP handling, business rules, persistence, and security.	"I separate concerns so each part has a clear reason to change and can be tested independently."
High cohesion, low coupling	Keep closely related behavior together and reduce unnecessary dependencies between components.	"I aim for focused components with small contracts, so a change in one area does not ripple across the service."
Composition over inheritance	Assemble behavior from components when a rigid parent-child hierarchy does not fit.	"I prefer composition for behavior that varies, such as payment methods, because I can replace or test each strategy independently."
Principle of least astonishment	Make an API behave as its name and contract suggest. Avoid hidden side effects.	"I design methods and endpoints so callers can predict their effects and error behavior from the contract."

Principle	Plain English	Professional interview line
Fail fast at boundaries	Reject invalid inputs early, with a clear error, before they can corrupt business state.	"I validate requests at the boundary and preserve deeper database constraints as a final defense for concurrent writes."

How they work together: KISS asks whether the current design is understandable. YAGNI asks whether a proposed feature is actually needed. DRY asks whether one business fact is maintained in several places. They can pull in different directions: extracting every repeated three-line block may increase complexity and couple unrelated rules. First confirm that the duplicated code represents the **same knowledge**, then decide whether sharing it makes future changes safer.

Payment example: Suppose two endpoints have similar validation code. Share a stable rule such as "amount must be positive" when both endpoints truly mean the same thing. Keep different authorization and retry rules separate if one endpoint creates a payment and the other reverses one. Do not build a universal payment workflow engine only because more payment types might arrive later.

A 30-second answer to memorize: "I use KISS to keep a solution understandable, YAGNI to avoid building for unconfirmed future needs, and DRY to keep each business rule in one place. I apply them together rather than mechanically. In a payment API, I would share a genuinely common amount rule, but keep different payment and reversal workflows separate when their failure behavior differs."

3. Collections and complexity [A]

Plain English: A List keeps ordered items and allows duplicates. A Set enforces uniqueness. A Map connects a key to a value. Choosing the right collection often turns slow code into simple code.

Say it professionally: "I choose collections from the access pattern. ArrayList provides constant-time indexed access and amortized constant-time append. HashMap usually gives constant-time lookup with good hashing, while TreeMap provides sorted keys with logarithmic operations. I also consider ordering, memory, and concurrency."

Know: ArrayList insertion in the middle is O(n); LinkedList indexed lookup is O(n), so it is not automatically faster. HashMap has no guaranteed iteration order. LinkedHashMap preserves defined iteration order. PriorityQueue exposes the smallest element by default; offer/poll are O(log n), peek O(1). ArrayDeque is a useful stack or queue. HashSet operations depend on hashing.

4. equals, hashCode and mutable keys [A]

Plain English: equals asks whether two objects represent the same value. hashCode helps a hash collection find the area where that value belongs. Equal objects must have the same hash code. Different objects may share a hash code.

Say it professionally: "When I define value equality, I implement equals and hashCode consistently. I use stable fields for map keys because changing a key's equality or hash after insertion can make the entry unreachable through normal lookup. A hash collision does not imply equality."

Follow-up: == compares object references; equals can compare logical values. Override both methods together. In a payment deduplication key, include tenant and the real business identifier. A mutable object's current hash can point to a different bucket from the one used at insertion.

5. Immutability, String and pass-by-value [A]

Plain English: An immutable object's value cannot change after construction. This makes sharing easier. Java passes a copy of a value into a method; for objects, that value is a reference. The method can mutate the referenced object, but reassigning its local reference does not replace the caller's reference.

Say it professionally: "I favor immutable value objects because they reduce accidental changes and simplify concurrency. I use defensive copies for mutable members. A final reference cannot be reassigned, but it does not make the referenced object immutable."

Know: String is immutable; StringBuilder is mutable and useful for repeated construction. Records are concise data carriers but only shallowly immutable. An unmodifiable view can still reflect changes in its backing collection. Know primitives versus wrappers, autoboxing, and null-unboxing failures.

6. Exceptions and resource management [A]

Plain English: An exception reports a failure. Handle it where you can make a useful decision: retry, translate, compensate, or return an appropriate error. Database connections and streams must be closed even when something fails.

Say it professionally: "I catch exceptions at boundaries where I can add context or choose a recovery action. I preserve the cause, avoid silently swallowing failures, and use try-with-resources for AutoCloseable resources. I distinguish validation failures from temporary dependency failures."

Follow-up: Checked exceptions require handling or declaration; unchecked exceptions do not. finally normally runs, but is not guaranteed after process termination. Never use return in finally. Do not log sensitive payment data or repeatedly log the same exception in every layer. Explain how an exception affects transaction rollback.

7. Generics and type safety [A]

Plain English: Generics let the compiler catch type mistakes. `List<Payment>` promises payment values. Wildcards allow flexibility when a method reads or writes related types.

Say it professionally: "I use generics to make contracts explicit and avoid unsafe casts. For flexible APIs, I use extends when a collection produces values and super when it consumes values. I avoid raw types because they move errors from compile time to runtime."

Know: `List<Integer>` is not a subtype of `List<Number>`. `? extends Number` allows reading as Number but not safely adding arbitrary Number values. `? super Integer` permits adding Integer values. Type erasure limits runtime access to generic type arguments.

8. Lambdas, streams and Optional [A]

Plain English: A stream describes steps for processing data. Intermediate steps such as filter and map are lazy; a terminal operation runs the pipeline. Optional represents a possibly missing result.

Say it professionally: "I use streams for clear transformations and aggregation. map changes each element, while flatMap combines nested results into one stream. I avoid shared mutable state in stream operations and use a loop when it makes control flow or debugging clearer."

Know: filter, map, flatMap, distinct, sorted, collect, groupingBy, reduce; Comparator.comparing; functional interfaces Predicate, Function, Consumer and Supplier. Streams are generally single-use. `orElse` evaluates its argument eagerly; `orElseGet` uses a supplier when needed. Parallel streams add overhead and shared-pool concerns; they are not a default performance optimization.

9. Money and modern Java features [A/B]

Plain English: Binary floating-point cannot represent many decimal amounts exactly. Financial calculations need an explicit decimal representation and rounding rules. New language features can simplify code, but only if the project's Java version supports them.

Say it professionally: "For monetary values, I use BigDecimal or an agreed integer minor-unit representation, with explicit currency, scale, and rounding rules. I avoid constructing BigDecimal from a binary floating-point value. I choose language features based on the supported runtime."

Know: Prefer `new BigDecimal("0.10")`; division can need a rounding mode. `new BigDecimal("1.0").equals(new BigDecimal("1.00"))` is false, while compareTo returns zero. Know Java 8 lambdas/streams/Optional, Java 17-era records/sealed classes, and Java 21 virtual threads. Do not guess which Java release the team uses.

4. Concurrency, JVM and Vert.x

10. Race conditions, atomicity and visibility [A]

Plain English: A race occurs when the result depends on the timing of concurrent operations. Atomicity means an operation cannot be observed half-complete. Visibility means one thread can see another thread's change.

Say it professionally: "I separate visibility from atomicity. volatile provides visibility and ordering guarantees for a field, but it does not make a read-modify-write operation such as increment atomic. I use an atomic operation or a lock depending on the invariant I need to protect."

Example: Two threads can both read 10 and both write 11 for `count++`. AtomicInteger.incrementAndGet prevents that lost increment. A transfer involving two balances needs a shared consistency boundary, not two unrelated atomic counters. Explain happens-before as a guarantee that earlier writes become visible through a defined synchronization relationship.

11. synchronized, locks and deadlocks [A]

Plain English: A lock lets one thread at a time protect shared state. A deadlock occurs when threads wait for each other forever. Long work while holding a lock makes other threads wait.

Say it professionally: "I keep critical sections small and establish a consistent lock order when multiple locks are needed. synchronized is a good default for simple mutual exclusion. I consider ReentrantLock when I need features such as timed acquisition or interruptible waiting, and release it in finally."

Follow-up: Instance synchronization locks that instance; static synchronization locks the Class object. Locks are reentrant. A concurrent container does not make a multi-step business workflow atomic. Inspect thread dumps for blocking and lock cycles; do not add more threads as the first response.

12. Concurrent collections and executors [A]

Plain English: ConcurrentHashMap supports concurrent access. However, "check then insert" is two operations unless you use an atomic method. An executor manages task execution, and its limits protect the application from overload.

Say it professionally: "I use operations such as putIfAbsent or compute when a per-key update must be atomic. I also bound concurrent work and queue growth, because an unbounded backlog can cause latency and memory failures even when the individual data structures are thread-safe."

Know: Thread, Runnable, Callable, Future, ExecutorService, bounded queues, rejection policies, shutdown, interruption. ConcurrentHashMap rejects null keys/values. Avoid slow remote work inside compute. `Executors.newFixedThreadPool` uses an unbounded work queue; a fixed thread count alone does not bound pending work. See Java concurrency API.

13. CompletableFuture and asynchronous composition [A]

Plain English: A future is a result that will arrive later. Chaining futures lets you describe dependent work without waiting after every step. Independent work can run concurrently if resources allow.

Say it professionally: "I use thenApply to transform a result and thenCompose when the next step also returns a future. I use thenCombine for independent results. I make executor choice, deadlines, and exception handling explicit so asynchronous code remains predictable."

Know: join/get wait for completion; avoid them on event-loop threads. An async stage without an explicit executor commonly uses the common pool. Non-async stages may run on the thread completing the prior stage. A future timeout does not necessarily stop the underlying I/O; configure client timeouts and cancellation where supported. Distinguish exceptionally, handle and whenComplete.

14. JVM memory, garbage collection and performance [B]

Plain English: The JVM manages object memory and reclaims unreachable objects. A memory leak still happens when the program keeps references to objects it no longer needs. High latency may come from CPU, waiting, memory pressure, or a dependency.

Say it professionally: "I start performance investigation with measurements. I compare latency, throughput, CPU, allocation, garbage collection, thread state, and dependency timing. I use thread dumps or profiling for execution problems and heap analysis for retained-memory problems before changing JVM settings."

Know: Heap, thread stacks, class metadata/metaspace, stack overflow versus out-of-memory, JIT compilation, allocation rate, retained size. Objects are generally heap-managed, with optimizations such as escape analysis. System.gc is not a reliable fix. An ever-growing map or retained listener can cause a leak. Increasing heap may postpone a failure without fixing the cause.

15. Virtual threads [B]

Plain English: Virtual threads make it cheaper to have many tasks waiting for I/O. They do not create extra CPU cores or make database connections unlimited.

Say it professionally: "Virtual threads can simplify blocking-style code for high-concurrency I/O workloads. I still limit access to scarce dependencies such as database connections. They improve scalability for suitable workloads, not the speed of CPU-intensive computation."

Version detail: Virtual threads became final in Java 21. Pinning behavior changed in newer JDKs; do not repeat Java 21 synchronization limitations as a universal rule for every version. Know the runtime before discussing those details. See Oracle's virtual-thread guide.

16. Vert.x event loops and worker execution [A]

Plain English: An event loop handles many small pieces of work. If one handler waits on a blocking call, other work assigned to that loop must wait too. Nonblocking I/O lets the loop do other work while waiting for a result.

Say it professionally: "Vert.x uses event loops to handle concurrent I/O efficiently. I keep event-loop handlers short and nonblocking. For unavoidable blocking work, I use worker execution or executeBlocking and control the workload so the worker pool does not become the next bottleneck."

Follow-up: Blocking JDBC, Thread.sleep, waiting on a future, and long CPU work can stall an event loop. Prefer an appropriate asynchronous client when available. A standard verticle's context simplifies concurrency, but shared state across contexts still needs care. Know verticles, handlers, Future/Promise, map/compose, routing and error handling. See Vert.x core guide.

17. Backpressure and event delivery [A/B]

Plain English: If data arrives faster than you can process it, a queue grows. Backpressure slows the producer or limits how much work you accept. Simply making code asynchronous does not solve overload.

Say it professionally: "I treat overload as an explicit behavior. I bound in-flight work, monitor queue depth, and pause, reject, or shed load when needed. I also distinguish in-process messaging from durable delivery requirements."

Follow-up: A Vert.x event bus is useful for communication, but should not be assumed to provide durable payment-event storage. For durable workflows, explain persistence, acknowledgement, retry and duplicate handling. For streams, understand pause/resume and drain handling. See Vert.x context guide.

5. Spring, APIs and tests

18. Spring dependency injection and bean scope [A]

Plain English: Spring creates and connects application components. Instead of creating a database client inside a service, the service receives it. This makes dependencies visible and tests easier.

Say it professionally: "I prefer constructor injection for required dependencies because it makes the contract explicit and supports straightforward unit testing. I keep singleton services stateless, since a singleton bean is shared and Spring does not automatically make its fields thread-safe."

Know: Component scanning; @Component, @Service, @Repository; @Configuration and @Bean; @Qualifier and @Primary; singleton versus prototype/request scope; bean initialization and destruction; circular dependencies. A Spring singleton is normally per application context and bean definition, not one global instance across every process.

19. Spring Boot and configuration [A]

Plain English: Spring provides the framework. Spring Boot helps assemble and configure an application with useful defaults. Those defaults depend on the libraries and configuration you provide.

Say it professionally: "Spring Boot reduces setup through starters and conditional auto-configuration. I still inspect the effective configuration, especially database pools, security, timeouts, and production endpoints. I externalize environment-specific values and keep secrets out of source control."

Know: @SpringBootApplication, profiles, configuration properties, validation, embedded server, health endpoints and graceful shutdown. Auto-configuration can back off when you supply your own beans; use the conditions report to understand why. See Spring Boot auto-configuration.

20. Request flow, validation and error handling [A]

Plain English: A request enters through security and routing, is validated, runs business logic, accesses data, and becomes a response. Each layer should have a clear job.

Say it professionally: "I keep controllers focused on HTTP concerns, services on business rules, and repositories on persistence. I validate request structure early, enforce authorization and business invariants in the appropriate layer, and translate failures into consistent responses with a correlation identifier."

Know: @RestController, mappings, DTOs, @Valid, @ControllerAdvice, filters/interceptors. Do not expose persistence entities by default or allow clients to set privileged fields. Syntactic validation is not authorization. Use parameterized queries, bounded pagination, stable error codes, and intentional logging.

21. Transactions and the proxy trap [A]

Plain English: A transaction groups database work so it succeeds or fails together. Spring commonly applies transaction behavior through a wrapper called a proxy. Calling a method directly within the same object bypasses that wrapper.

Say it professionally: "I place transaction boundaries around cohesive business operations and keep them short. In Spring's default proxy mode, self-invocation bypasses transactional advice. I also check rollback rules explicitly: the usual default rolls back for runtime exceptions and errors, not every checked exception."

Follow-up: REQUIRED joins an existing transaction or starts one. REQUIRES_NEW uses an independent transaction and can need another connection. Catching and swallowing an exception can change rollback behavior. Thread-bound transactions do not automatically follow newly created threads. Configuration can override defaults. See Spring transaction annotations and transaction implementation.

22. Persistence, JPA and the N+1 problem [A/B]

Plain English: An object mapper can hide SQL details, but the database still executes queries. N+1 means one query loads a group of rows and additional queries load related data one item at a time.

Say it professionally: "I inspect the SQL generated by the persistence layer. For N+1 behavior, I choose a fetch join, entity graph, batch fetch, or projection based on the use case. I verify query count and pagination behavior instead of changing every relationship to eager loading."

Know: Entity lifecycle, persistence context, dirty checking, lazy loading, flush versus commit, optimistic @Version locking. Flush sends changes toward the database; it is not the same as committing the transaction. Spring Data is not synonymous with JPA: know whether your project uses JPA, JDBC, or another repository technology.

23. HTTP and REST contracts [A]

Plain English: HTTP methods describe the intended operation. A good API tells clients what happened and what to do next. Idempotency means repeating an operation has the same intended effect as doing it once.

Say it professionally: "I define API behavior around resources, validation, authorization, and stable contracts. GET should be safe; PUT and DELETE are idempotent in their intended effect. POST can be made retry-safe with an explicit idempotency contract. I do not equate identical responses with idempotency."

Know: GET/POST/PUT/PATCH/DELETE; PUT replacement versus PATCH partial update; PATCH is not inherently idempotent. 200 success, 201 creation, 202 accepted for later processing, 204 no body, 400 invalid request, 401 missing/invalid authentication, 403 forbidden, 404 not found, 409 conflict, 429 rate limit, 500 internal error, 503 unavailable. Explain pagination, backward-compatible changes, versioning, TLS and connection reuse.

24. Testing and code quality [A]

Plain English: Different tests find different mistakes. Unit tests check local logic. Integration tests check real interactions. Contract tests check agreements between services. A small set of end-to-end tests checks complete flows.

Say it professionally: "I choose tests around failure risk. I unit-test business rules, use integration tests for database constraints and transaction behavior, and verify API contracts. For payment operations, I explicitly test duplicate requests, concurrent updates, timeout ambiguity, and authorization boundaries."

Know: JUnit assertions and parameterized tests; Mockito stubbing and verification; test doubles versus real dependencies. Do not mock away the behavior you need to verify. Prefer deterministic concurrency tests with coordination primitives over arbitrary sleeps. High coverage alone does not prove useful assertions. Static analysis complements review and testing.

6. Databases and caching

25. SQL fundamentals and data modeling [A]

Plain English: Tables hold related facts. Keys identify rows and relationships. SQL combines, filters, groups, and orders those facts. Constraints protect rules even when multiple application instances write at once.

Say it professionally: "I use primary, foreign, and unique constraints to protect data integrity, and I choose joins and aggregates from the business question. I pay attention to NULL semantics and row multiplication in joins because a query can be syntactically correct but financially wrong."

Know: INNER/LEFT JOIN, WHERE/HAVING, GROUP BY, COUNT(*) versus COUNT(column), SUM, subqueries, CTEs, ROW_NUMBER and DENSE_RANK. Use IS NULL. A filter on the right table in WHERE can unintentionally remove unmatched LEFT JOIN rows. Joining payments to multiple allocation rows can double-count payment totals.

26. Indexes and slow-query diagnosis [A]

Plain English: An index is an extra data structure that helps find rows. It costs storage and write work. An index helps only if the query and data distribution make it useful.

Say it professionally: "I start with the execution plan, actual row counts, and query pattern. I consider composite-index order, selectivity, sorting, and coverage. I then measure the read improvement against the extra cost on inserts and updates."

Example: For tenant-scoped queries filtered by status and ordered by time, consider `(tenant_id, status, created_at, id)` and verify the actual plan. Avoid claiming it is automatically optimal. Functions or implicit casts on indexed columns can prevent useful access paths. Check statistics, locks, connection-pool waiting, and query count before blaming the database engine.

27. ACID, isolation and lost updates [A]

Plain English: Atomicity is all-or-nothing. Consistency means rules remain valid. Isolation controls interference between concurrent transactions. Durability means committed work survives under the database's configured guarantees.

Say it professionally: "I choose isolation and locking based on the business invariant. I use unique constraints and atomic updates where possible. For concurrent edits, optimistic locking detects a stale version; pessimistic locking prevents conflicting access by making transactions wait. I plan for deadlock or serialization retries."

Know: Dirty read = uncommitted data; nonrepeatable read = the same row changes between reads; phantom = matching row set changes. Read committed prevents dirty reads; stronger levels constrain more anomalies, with engine-specific behavior. Serializable targets the effect of serial execution but may abort transactions. Verify DB2, Oracle or MySQL semantics rather than assuming they match another engine. Isolation reference and examples.

28. Redis and cache correctness [A/B]

Plain English: A cache keeps a copy of data for faster reads. The database remains the authority in a cache-aside design. Cached data can become old, and many simultaneous misses can overload the database.

Say it professionally: "I use cache-aside for suitable repeated reads, with tenant-aware keys, expiration, and an invalidation strategy. I measure hit rate and tail latency, and plan for cache failure and stampedes. I do not treat cached permissions or payment state as automatically authoritative."

Know: Read cache -> on miss query database -> populate cache. After a committed write, invalidate or update according to the consistency design. TTL limits an entry's lifetime but does not eliminate stale-write races. Use versioning, coordinated refresh, or stronger reads when required. TTL jitter and request coalescing can reduce stampedes. Eviction differs from expiry. A cache outage needs bounded fallback, not unlimited database traffic. Redis cache-aside guide.

29. Couchbase and SQL versus document storage [B]

Plain English: A document database stores related information in documents. It can suit flexible records and key-based access. You still need deliberate schemas, indexes and concurrency control.

Say it professionally: "I choose storage based on access patterns and consistency requirements. In Couchbase, I distinguish key-value operations from indexed SQL++ queries. I use CAS for optimistic concurrency and select query consistency deliberately when recent writes must be visible."

Know: CAS changes when a document changes; a mismatched CAS rejects a stale mutation. Secondary indexes may lag writes. request_plus waits for indexing through the request's consistency point; it does not promise inclusion of every later concurrent write. Do not say all NoSQL databases lack transactions. See Couchbase CAS and SQL++ consistency.

7. Security and payment correctness

30. Authentication, authorization, IAM and MFA [A]

Plain English: Authentication establishes who you are. Authorization decides what you may do. MFA strengthens authentication with independent factor types. A valid login does not grant access to every account.

Say it professionally: "I separate identity verification from access decisions. I validate the principal, then enforce tenant, resource, and action-level authorization on every sensitive operation. RBAC provides role-based permissions, while contextual policies can handle additional restrictions. I follow least privilege and audit important access changes."

Follow-up: A password and another password are not two different factor types. Explain enrollment, recovery, lockout, session expiry, deprovisioning and permission revocation. For `/accounts/{id}`, never rely only on possession of the ID. Test cross-tenant access and privilege escalation. OWASP object-level authorization.

31. OAuth, OpenID Connect and JWT [A/B]

Plain English: OAuth delegates access. OpenID Connect adds an identity layer. JWT is a token format, not a login protocol. A signed token can be readable even though changes to it can be detected.

Say it professionally: "I distinguish the authorization protocol from the token format. For a JWT, I verify the signature with trusted keys, restrict accepted algorithms, and check issuer, audience, time claims, and token purpose. I do not assume a valid signature alone is enough."

Know: Access token versus ID token, scopes versus application permissions, expiry, key rotation, refresh-token handling. Do not accept an ID token as an API access token without an explicit appropriate contract. Authorization code with PKCE is relevant for user-facing clients; client credentials is for suitable service-to-service access. Explain revocation needs. JWT security guidance.

32. API security beyond login [A]

Plain English: An authenticated client can still send malicious input, request another customer's record, or overwhelm a service. Security checks must match each operation.

Say it professionally: "I validate inputs, parameterize database access, enforce authorization on server-side resources and properties, and bound request size and rate. I protect secrets, minimize sensitive logging, and test negative cases alongside the successful flow."

Know: Injection, broken object authorization, mass assignment, server-side request forgery, dependency vulnerabilities and sensitive error messages. Allowlist writable fields. Use destination restrictions for server-side fetches. SAST inspects code; DAST exercises a running application; dependency scanning identifies known component issues. None proves the application is secure by itself.

33. Message signing and certificate renewal [A/B]

Plain English: Encryption protects confidentiality. A digital signature helps verify the signer and detect modification. Certificate renewal replaces expiring identity material; it must also reach every system that depends on it.

Say it professionally: "For certificate automation, I treat renewal as a lifecycle: inventory, expiry detection, issuance, secure deployment, trust propagation, verification, and alerting. Where supported, I allow a controlled overlap between old and new certificates and retain a rollback path. I verify use of the new certificate before retiring the old one."

Follow-up: Distinguish TLS certificates from the message-signing certificates named in your resume. Explain keystore versus truststore, public/private keys, certificate chain, renewal versus key rotation, and failure recovery. State the tooling you actually used. Avoid exposing private keys in logs or repositories. Your resume does not establish which certificate authority, scheduler or secret store you used.

34. Idempotency and duplicate payment requests [A]

Plain English: A client may retry because the response was lost even though the payment was accepted. The service must recognize that retry instead of creating another payment.

Say it professionally: "I make retry safety an explicit contract. I scope an idempotency key to the caller or tenant, bind it to the request, and enforce uniqueness in durable storage. Repeated matching requests return the existing outcome, while conflicting payloads are rejected. I also handle concurrent duplicates and in-progress work."

Follow-up: Persist key, payload fingerprint, status and result/reference with a defined retention policy. A local map fails across instances and restarts. A Redis-only key with expiry does not necessarily guarantee durable financial deduplication. A database record alone does not deduplicate an external charge: use provider idempotency and reconciliation when the provider outcome is uncertain.

35. Timeouts, retries, messaging and partial failure [A/B]

Plain English: A timeout means you did not receive a result in time. It does not prove that the other system did nothing. Retrying can help temporary failures but can also repeat side effects or worsen overload.

Say it professionally: "I set deadlines and retry only failures that are transient and safe to repeat. I use bounded attempts with backoff and jitter. For an uncertain payment outcome, I reconcile using a stable reference rather than blindly submitting a new charge."

Know: Circuit breaker avoids repeated calls to a failing dependency; bulkhead limits the resources one dependency can consume. At-least-once delivery requires duplicate-safe consumers. An outbox stores a business change and event record in one local transaction; a relay publishes later and may publish duplicates. It does not make an external payment atomic. Transactional outbox reference.

8. Production skills and brief secondary review

36. Logs, metrics, traces and incident response [A/B]

Plain English: Logs describe events, metrics show trends, and traces connect steps in a request. A slow request can spend most of its time waiting in a pool or dependency rather than executing your code.

Say it professionally: "I first assess customer impact and recent changes, then use latency, errors and saturation to narrow the problem. Distributed traces show where time is spent, while correlated logs provide context. I mitigate safely, verify recovery, and follow with root-cause analysis and prevention."

Know: p50/p95/p99, throughput, error rate, queue depth, database pool usage, event-loop delay, GC pauses. Average latency can hide a bad tail. Propagate trace context across service boundaries. Use Jaeger and Splunk examples only to the depth you actually know. Distinguish trace IDs from business IDs; avoid secrets and high-cardinality metric labels.

37. CI/CD, blue-green and canary releases [B]

Plain English: A delivery pipeline makes changes repeatable. Blue-green switches traffic between two environments. Canary sends a limited amount of traffic to a new version while you watch the result.

Say it professionally: "I combine automated checks with a deployment strategy that limits exposure. I define health and business signals for promotion or rollback, and keep database and API changes compatible across versions. A rollback plan must account for data changes, not just application binaries."

Know: Build/test/scan/package/deploy/verify; immutable artifacts; Git review and revert; feature flags; expand-and-contract schema changes. A green health endpoint alone does not prove business correctness. Review error rate, tail latency, duplicate transactions and authorization failures where relevant.

38. Reliability, DR and Kubernetes basics [B/C]

Plain English: High availability keeps a service running through failures. Disaster recovery restores it after a larger disruption. Recovery time and acceptable data loss are separate targets.

Say it professionally: "I define recovery objectives and test the recovery process, including data integrity and dependency readiness. For containers, I distinguish readiness to receive traffic from liveness that determines whether a restart is needed. I avoid restart behavior that amplifies a shared dependency outage."

Know: RTO = target recovery time; RPO = target data-loss window; SLI = measured indicator; SLO = target; SLA = service agreement. Startup probes protect slow startup, readiness removes unready pods from service routing, and liveness can trigger restart. Know Pod, Deployment, Service, requests/limits, autoscaling and graceful shutdown. Kubernetes probe guide.

39. AWS, Node.js and integration technologies [C]

Plain English: Learn what the named technology does and how it relates to your work. Do not attempt a cloud-certification syllabus before this coding interview.

Say it professionally: "My strongest hands-on area is [your actual area]. For [secondary technology], my experience is [implemented / reviewed / conceptual]. I can explain its role and tradeoffs, and I would confirm service-specific configuration before making a production decision."

Quick review: AWS identity roles, network boundaries, load balancing, compute, object storage, managed databases and monitoring. Node.js uses an event loop for JavaScript callbacks; asynchronous APIs and worker facilities handle other work, but long CPU callbacks can block progress. SOAP uses XML messaging/contracts; REST is an architectural style, commonly implemented with HTTP. Spring Integration provides message-oriented components such as channels, routers, transformers and adapters. For EDI/ETL, review validation, mapping, acknowledgements, control totals, duplicates, checkpoints and safe reprocessing from your actual projects.

40. Airflow and Spark [C]

Plain English: Airflow schedules and coordinates tasks. Spark processes data across workers. The scheduler and the data-processing engine solve different problems.

Say it professionally: "I distinguish workflow orchestration from distributed processing. I design retried pipeline tasks to be idempotent and tied to a specific input partition. In Spark, I pay attention to shuffles and skew because data movement and uneven partitions can dominate runtime."

Know: Airflow DAGs, dependencies, retries and backfills; pass references to large data rather than large payloads between tasks. Spark transformations are lazy, actions trigger computation, and shuffles redistribute data. Review only if asked or if you choose these technologies as experience examples. See Airflow practices and Spark programming guide.

9. Coding workbook: practice in this order

Use a blank editor. Before coding, state input assumptions, output, an example, a simple approach, and a better approach. Then write code, trace it, test boundaries, and state complexity. If stuck after 8 minutes, read only the hint. After reviewing a solution, repeat from a blank editor later.

The companion file **InterviewPractice.java** contains reference implementations for all ten exercises and executable checks. Try the questions before reading it. The examples use Java 8-compatible syntax; their comments define assumptions such as lowercase words, UTF-16 characters, interval boundaries, and already-deduplicated payment input. Run `javac InterviewPractice.java` followed by `java InterviewPractice` in a folder containing the file and a configured JDK.

#	Exercise and timebox	Expected approach and complexity	Essential tests
1	Two Sum: return two different indices whose values sum to a target. 20 min	Hash map of seen values; expected O(n) time/O(n) space	Duplicate values, negative values, no match, fewer than 2 items, integer overflow in complement
2	Group anagrams of lowercase English words. 25 min	Sorted key: O(total word-sorting cost), or 26-count signature: O(total characters + 26n)	Empty input, empty word, duplicates, clarify case and character set
3	Longest substring without repeated characters. 30 min	Sliding window with last-seen positions; O(n) time	Empty, all identical, repeated pattern, define UTF-16 units versus Unicode code points

#	Exercise and timebox	Expected approach and complexity	Essential tests
4	Merge overlapping closed intervals. 30 min	Sort by start then scan; O(n log n) time	Nested intervals, touching endpoints, unsorted input, empty input, invalid intervals
5	First occurrence of a target in a sorted array. 20 min	Binary search with candidate answer; O(log n) time/O(1) space	Repeated target, absent target, empty array, first and last positions
6	Validate balanced brackets. 20 min	Stack using ArrayDeque; O(n) time/O(n) space	Empty string, wrong nesting, early close, leftover opens, non-bracket characters policy
7	Top K frequent payment categories. 30 min	Count map plus heap of size k; O(n + u log k), u unique categories	k=0, k larger than u, ties with deterministic rule
8	Binary-tree level-order traversal. 25 min	Breadth-first traversal with queue; O(n) time, O(width) auxiliary space	Null root, single node, skewed tree, balanced tree
9	LRU cache get/put in O(1). 35 min	Hash map + doubly linked list; or access-order LinkedHashMap if allowed	Capacity 1, update existing key, eviction order, invalid capacity
10	Aggregate successful payments by account and currency. 25 min	Loop or streams with BigDecimal; O(n) expected grouping time	Failed/pending excluded, multiple currencies, scale, null policy, duplicate event policy

Core pass: exercises 1-6 and 10. Use exercises 7-9 for later practice or if you finish early. Do not spend three hours on advanced dynamic programming while basic Java implementation remains uncertain.

Correctness explanations to practice

Two Sum: "Before each iteration, the map contains indices from earlier elements. I look for the required complement before inserting the current value, so I never use the same index twice."

Sliding window: "The current window contains no repeated character. When a repeat appears inside the window, I move the left boundary past its previous occurrence. The left boundary never moves backward."

Merge intervals: "After sorting by start time, the next interval either overlaps the last merged interval or starts a new one. I extend the end using the maximum end so nested intervals remain correct."

Binary search: "I preserve the range that can still contain the first occurrence. When I find the target, I save its position and continue searching left."

LRU: "The map gives direct lookup, and the list maintains recency. Every successful access moves that entry to the most recent position. Eviction removes the least recent entry from both structures."

Exercise 10: payment aggregation contract

Assume each record contains accountId, currency, amount and status. Group only SUCCESS records by **account and currency**, because adding different currencies without conversion is meaningless. Decide whether input records are already deduplicated; do not silently assume transaction IDs are unique. Use BigDecimal addition and an explicit null policy. If the interviewer asks for parallel processing, explain associativity and thread-safe accumulation before changing the implementation.

Three useful code-review drills

Drill A - race: `if (!map.containsKey(key)) { map.put(key, value); }` is not atomic merely because map is concurrent. For an in-memory per-key insert, use putIfAbsent. For a cross-instance business guarantee, use durable coordination such as a unique database constraint.

Drill B - event loop: A Vert.x handler calls a synchronous JDBC query, then `future.join()`. Both can block. Explain asynchronous composition or bounded worker execution, and how you would test event-loop responsiveness under load.

Drill C - transaction: A method calls another @Transactional method through `this`, catches its exception, and returns success. Check proxy bypass, the actual transaction boundary, rollback rules and error propagation. Do not fix it by adding more annotations without understanding the call path.

10. SQL workbook

Assume `payments(id, tenant_id, account_id, currency, amount, status, created_at)` and `accounts(id, tenant_id, name)`. The exercises assume `(tenant_id, id)` identifies an account, amounts use a decimal type, and timestamps are comparable. Adapt timestamp and pagination syntax to the interview database.

Q1. Successful totals by account and currency

Write the query first, then compare:

```
SELECT tenant_id, account_id, currency,
       COUNT(*) AS payment_count, SUM(amount) AS total_amount
FROM payments
WHERE status = 'SUCCESS'
GROUP BY tenant_id, account_id, currency;
```

Explain: WHERE filters individual rows before grouping. HAVING filters grouped results. Do not combine currencies or omit tenant boundaries. Clarify how refunds and reversals appear in the data.

Q2. Accounts with no successful payment

```
SELECT a.tenant_id, a.id, a.name
FROM accounts a
WHERE NOT EXISTS (
  SELECT 1
  FROM payments p
  WHERE p.tenant_id = a.tenant_id
    AND p.account_id = a.id
    AND p.status = 'SUCCESS'
);
```

Explain: NOT EXISTS expresses the absence of a matching row and avoids the NULL trap of NOT IN when the subquery can return NULL. An account with failed payments only should still appear.

Q3. Latest payment per account

```
WITH ranked AS (
  SELECT p.*,
         ROW_NUMBER() OVER (
           PARTITION BY tenant_id, account_id
           ORDER BY created_at DESC, id DESC
         ) AS rn
  FROM payments p
)
SELECT id, tenant_id, account_id, currency, amount, status, created_at
FROM ranked
WHERE rn = 1;
```

Explain: The extra id ordering makes timestamp ties deterministic. Decide whether the requirement means latest payment of any status or latest successful payment; filter at the appropriate stage. ROW_NUMBER chooses one row; DENSE_RANK can keep ties.

Q4. Detect duplicate business requests

Assume an additional `idempotency_key` column:

```
SELECT tenant_id, idempotency_key, COUNT(*) AS occurrences
FROM payments
WHERE idempotency_key IS NOT NULL
GROUP BY tenant_id, idempotency_key
HAVING COUNT(*) > 1;
```

Explain: This detects existing duplicates; it does not prevent concurrent insertion. Prevention requires an appropriate unique constraint and transaction handling. Null uniqueness behavior differs by database, so define whether keys are required.

Q5. Prevent a lost update

Assume `account_balances(tenant_id, account_id, balance, version)`:

```
UPDATE account_balances
SET balance = balance - :amount, version = version + 1
WHERE tenant_id = :tenant
  AND account_id = :account
  AND version = :expected_version
  AND balance >= :amount;
```

Explain: Require a positive amount, check the affected-row count, and place related ledger writes in the same appropriate transaction. Zero updated rows can mean stale version, insufficient balance, or missing account; resolve deliberately. This statement alone is not a complete payment or transfer implementation.

11. Project stories you should prepare

Use the same five-part structure: **problem -> your responsibility -> implementation -> tradeoff/failure handling -> evidence and learning**. Spend most of the time on what you personally did. Say "we" for team outcomes and "I" for your contribution.

Story A: Java/Vert.x user-management APIs and Redis

Professional opening to adapt: "One relevant project was our commercial user-management platform. My role included [your exact backend responsibility]. The main challenge was [specific problem]. We used Java and Vert.x for the services, and I contributed to [specific implementation]. For frequently accessed data, we introduced Redis caching and measured [verified metric]. The main tradeoff was [freshness, complexity, or another actual concern], which we handled through [actual approach]."

Prepare the details: Trace one endpoint from authentication to response; identify the source of truth; name the client libraries and data model you actually used; explain worker versus event-loop behavior; define cache keys, invalidation and TTL policy; show how a permission change becomes effective; explain tests and the performance measurement.

Questions to rehearse: Why Vert.x? What exactly was cached? Why not cache every response? What happens when Redis is down? What was the baseline for 35%? What did you code yourself? How did you test tenant isolation?

Full technical mock for hour 18

Minutes	Prompt	What a strong answer demonstrates
0-5	Introduce yourself and describe a recent backend contribution	Clear personal ownership and relevant technical depth
5-12	HashMap equality; volatile versus atomic increment; one stream operation	Correct mechanism and a small example
12-37	Solve merge intervals or a fresh hashing problem	Clarification, correct code, invariant, tests, complexity
37-45	SQL: latest successful payment per account	Correct filtering, partitioning, tie handling and tenant scope
45-53	A Vert.x API slows under load; retries create duplicate requests	Evidence-based diagnosis, event-loop care, safe retry semantics
53-58	Explain one project decision and a failure you handled	Truthful ownership, tradeoff, validation and learning
58-60	Ask the interviewer two questions	Interest in the team's actual engineering work

Score yourself out of 20

Give 0-4 points each for **coding correctness, complexity/reasoning, edge-case tests, technical accuracy, and clear communication/ownership**. A 0 means missing; 2 means partly correct with significant prompting; 4 means independent and precise. This is a personal practice rubric, not American Express's hiring rubric.

Target 15+ with no zero, then repair the weakest category. A confident wrong explanation is a bigger problem than a short pause followed by a correct answer.

Rapid oral drill: 15 questions

1. Why must equals and hashCode agree?
2. Why is volatile count++ unsafe?
3. When does ConcurrentHashMap still need higher-level coordination?
4. What is the difference between thenApply and thenCompose?
5. Why can JDBC block a Vert.x event loop?
6. What does Spring Boot add to Spring?
7. Why might @Transactional not take effect?
8. How do you detect and fix N+1 queries?
9. How do you prevent lost updates?
10. Why can a LEFT JOIN unexpectedly remove unmatched rows?
11. How do you invalidate Redis data safely enough for the use case?
12. What must you validate in a JWT besides its signature?
13. How do you prevent cross-tenant access?
14. What does a payment timeout actually tell you?
15. What did you personally implement in your strongest project?

Answer each in 30-60 seconds, then take one follow-up. If an answer sounds vague, add a small example instead of more terminology.